Rapport et tache déploiement :

Rapport de déploiement — Intégration MQTT / Crow / MySQL

Date : 15–16 octobre 2025

Rédigé par : Ayoub smayen

1. Objectif du projet

L'objectif de cette phase est d'intégrer une messagerie MQTT au sein d'une application C++ basée sur le framework Crow (API REST) et la base de données MySQL.

Intégrer la messagerie MQTT dans une application C++ basée sur Crow et MySQL pour recevoir/émettre des messages d'appareils
Cette intégration permettra de :

- Recevoir et émettre des messages en provenance d'appareils connectés (IoT) via MQTT.
- Stocker les messages entrants dans la base MySQL.
- Fournir des endpoints HTTP pour déclencher l'envoi ou la lecture des messages.
- Mettre en place des outils de diagnostic et de supervision, tels que tcpdump et les journaux applicatifs.

Exemple de message MQTT reçu :


```
{
  "IDCodeAppareil": "Q04555581",
  "IDClient": "99999990",
  "arg": 1310,
  "IDPARTITION": 3,
  "timestamp": "2025-10-21T14:12:00Z"
}
```


Fournir endpoints HTTP pour déclencher envoi MQTT, stocker messages entrants en base, et fournir outils de capture/diagnostic (tcpdump) — output attendu : rapport, code, captures EQMx.

2) Plan de déploiement « en temps par action » (tableau)

Action Responsable Durée estimée Début Fin Dépendances

1. Préparation environnement (OS, cmake, MySQL, eqmx) Ops / Dev : 2 j
2. Installer et tester broker MQTT (mosquitto) 1 j
3. Intégration socket MQTT client dans projet C++ / CMake Dev 7 j
4. Ecoute mqqt eqmx server et lancer commande AXPR : 3 j
5. Endpoints Crow : publish, subscribe handler, select client ID Dev : 5 j
6. Persistance MySQL (insertion messages, indexation) Dev DB : 1 j
7. Tests end-to-end (simuler appareils, capture EQMx/mqtt) QA : 4 j
8. Logging / monitoring (tcpdump, journald, fichier capture) Dev/Ops : 3 j
9. Mise en prod & rollback plan Ops : 2 j
10. Documentation & rapport final (code + captures EQMx) Dev : 5 j

3) Architecture logicielle (bref)

3.1. Composants

- Crow Framework
  Fournit une API REST pour :
  - POST /publish → Publier un message MQTT
  - GET /messages?client=99999990 → Lire les messages du client
  - GET /status → Vérifier la santé du système
- Client MQTT (C++)
  - Connexion au broker (port 8883, protocole TLS)
  - Authentification par user/password ou certificat X.509
  - Souscription aux topics (ex. clients/+/commands)
  - Publication sur des topics définis (ex. clients/<ID>/status)
- Base de données MySQL
  - Table principale : messages
  - Champs : id, id_client, id_appareil, arg, id_partition, timestamp, payload
  - Indexation sur id_client et timestamp
- Broker MQTT (EQMX)
  - Utilisé en phase de test : EQMX (light)

Crow : fournit API REST (ex: POST /publish, GET /messages?client=99999990).

Socket MQTT C/C++ : client MQTT pour connect/publish/subscribe.

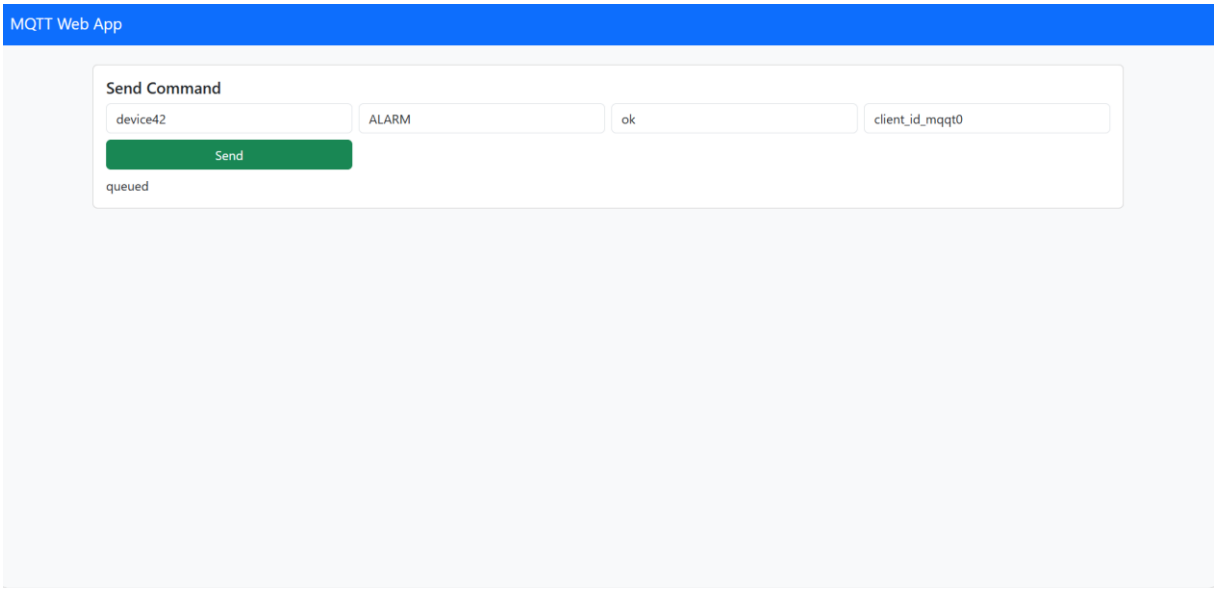
MySQL : persistance des messages.

Broker : eqmx (pour tests) — en production choisir broker robuste (EMQX).

En dessous example demo version 1 :

Example :

Demo 1 : CROW + socket eqmx C++



Receiver

MQTT Web App

Last Received (executed)


```
{
  "who": "appdemo",
  "command": "ALARM",
  "client_id": "client_id_mqqt0",
  "status": "ok",
  "id": "device42"
}
```


Manual Execute (true) | Manual Execute (false)

MQTT Web App

Sender | Receiver

Execution Trace

Auto-executes the **last** queued sender message.

Refresh

#	Time	Who	Action	ID	Command	Exec
1	2025-10-15 20:38:09	sendrec	response_sent		{"id":"device42","execute":"true","client_id":""}	?
2	2025-10-15 20:38:09	receiver_manual	manual_response	device42		?
3	2025-10-15 20:37:09	sendrec	response_sent		{"id":"device42","execute":"true","command":"ALARM","status":"ok","client_id":"client_id_mqqt0"}	?
4	2025-10-15 20:37:09	receiver	executed	device42	ALARM	?
5	2025-10-15 20:37:09	publisher	published	device42	ALARM	
6	2025-10-15 20:37:08	sender	sent	device42	ALARM	

Last Published Message


```
{
  "who": "appdemo",
  "command": "ALARM",
```


Last Published Message


```
{
  "who": "appdemo",
  "command": "ALARM",
  "client_id": "client_id_mqqt0",
  "status": "ok",
  "id": "device42"
}
```

Mise a jour

Pour message entre server et client reponse

Exemple concret de message JSON (payload)


```
{
  "IDCodeAppareil": "Q04555581",
  "IDClient": "99999990",
  "arg": 1310,
  "IDPARTITION": 3,
  "timestamp": "2025-10-21T14:12:00Z"
}
```


Utiliser TLS pour broker (port 8883).

Authentifier clients (user/password ou certificats).

Limiter ACL topics (ex: clients/+/commands).

Mettre en place monitoring (EMQX / mosquitto_exporter).

Rotation des logs et purge des tables MySQL si volumétrie élevée

Conclusion

Ce déploiement vise à démontrer une intégration stable et sécurisée entre messagerie MQTT, API REST Crow, et persistence MySQL.

L'ensemble servira de base pour les futures versions (v2) intégrant :

Interface web de supervision en temps réel,

Tableau de bord des appareils,

Historique et analyse des événements MQTT.